

Compiler Backend / Code Generation Engineer with professional LLVM 22/C++ experience at MCST. Practical proof points: SSA/CFG validation, liveness/interference, register allocation and spills, phi lowering, SysV x86-64 emission, and native-vs-interpreter differential execution.

2022: Compilers / VMs · 2023: CIL + native x86-64 · 2024–2025: AST / IR / backends → toolchain · 2026 → : Professional LLVM + static analysis

PROFESSIONAL EXPERIENCE

SKILLS

- 2026 — present

ISP RAS — Static Analysis Engineer

 - Contribute to SharpChecker, ISP RAS's C#.NET static-analysis platform.
 - Current work includes symbolic-computation and symbolic-execution tasks in C#.NET program analysis.
- July — August 2026

MCST — Compiler Engineering Intern

 - Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.
 - Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.

Backend / Codegen
x86-64, SysV ABI, stack frames, calling conventions, liveness/interference, register allocation, spills, phi lowering, native execution

Low-level / Assembly
Iced.Intel code emission, executable-memory JIT experiments; NASM IA-32, CDECL, x87

LLVM / IR
C++23, LLVM 22/IR, CFG/SSA, loop/interprocedural analysis, IR transformations

Verification
assignment verifier, LLVM Verifier, structural checks, idempotence, differential execution

PROJECTS

- x86-64 Compiler Backend & Register Allocation**
SSA/CFG validation → liveness/interference → register allocation/spills → phi lowering → SysV x86-64 emission.
Assignment verifier plus native-vs-interpreter differential execution; the current project CI is green; a separate public native-codegen lineage starts with Visk (2023) and Wist4 (2024).
- LLVM Interprocedural Global IV Optimization**
An LLVM 22 pass localizes supported induction-like globals after interprocedural effect analysis, affine evolution, and a separate applicability check; APInt models fixed-width wraparound.
29 positive/negative regression cases cover LLVM Verifier checks, structural expectations, idempotence, and observable before/after behavior.
- UniversalToolchain**
Dependency/conflict resolution, implementation selection, pass ordering, and typed artifact routes before execution.
1,324/1,324 tests; interpreter/CIL parity and configuration checks before runtime.

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

- LangDev'26 — Build the Language, Then Make the Abstractions Disappear**
Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.
- UniversalToolchain — Baltic Science and Engineering Competition**
First Degree Diploma and Grand Prize "Perfection as Hope" for UniversalToolchain.
- MEPhI Junior**
Overall winner; First Degree Diploma, 96/100 for UniversalToolchain.

Moscow / remote